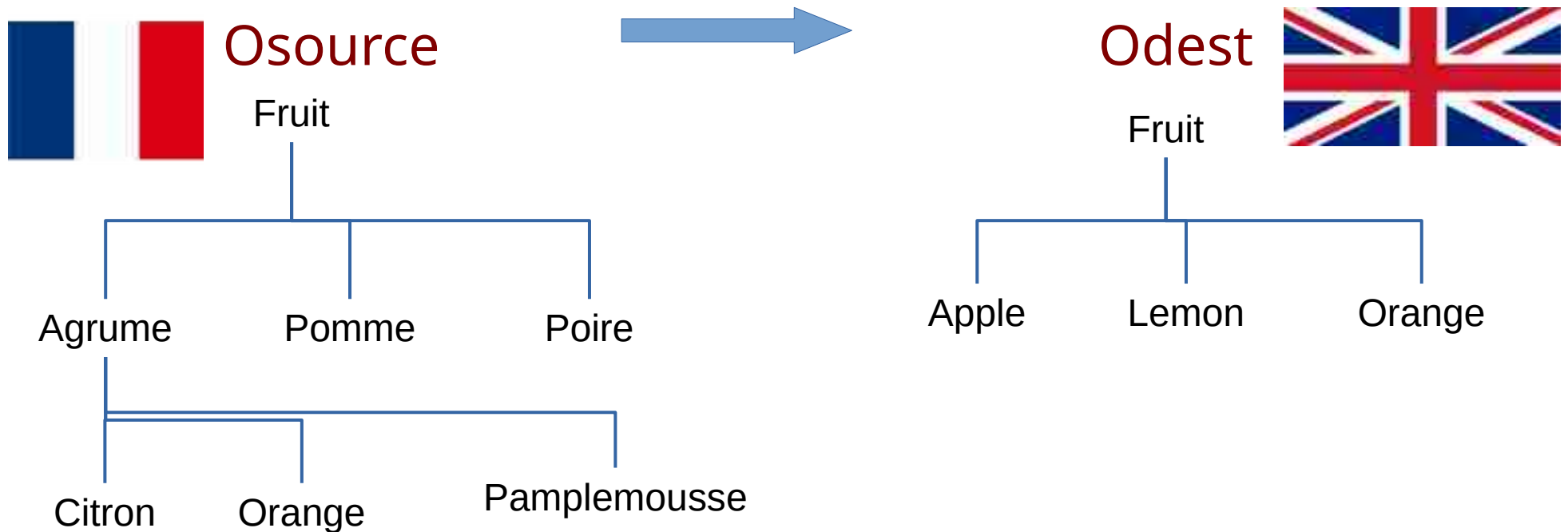


Relazioni fra ontologie

- **Identical**: O1 e O2 sono la stessa ontologia (esempio: un'ontologia e la sua copia in un disco mirror)
- **Equivalent**: condividono vocabolario e assiomatizzazione ma sono espresse in linguaggi differenti (esempio: SKOS e RDF)
- **Extension**: O1 estende O2 quando tutti i simboli definiti in O2 sono preservati in O1 insieme alle loro proprietà e relazioni ma non vale il viceversa
- **Weakly-Translatable**: siano Osource e Odest due ontologie, è possibile tradurre espressioni Osource in espressioni Odest con perdita di informazione
- **Strongly-Translatable**: slide
- **Approx-Translatable**: slide

Weakly-Translatable, esempio



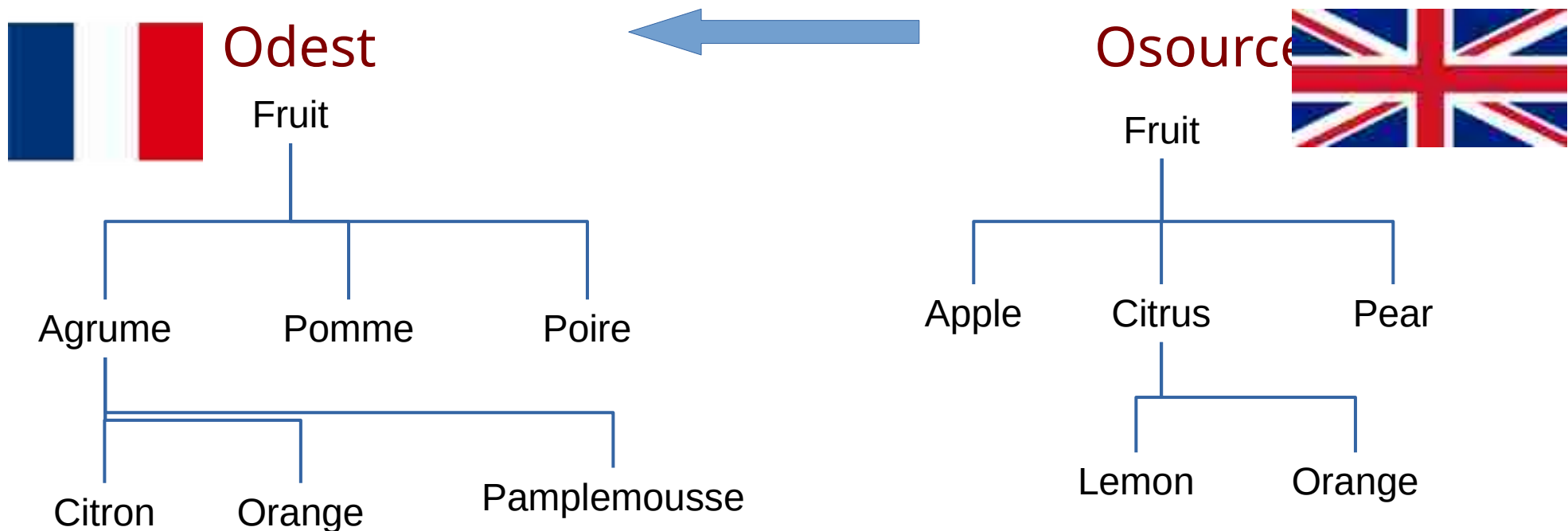
Osource è weakly-translatable in Odest usando il mapping:
Pomme → Apple, Citron → Lemon, Orange → Orange, Fruit → Fruit

Si ha perdita di informazione relativa a Poire, Pamplemousse e alla sottocategorizzazione di Agrume in Citron, Orange e Pamplemousse

Strongly-Translatable

- Osource è strongly-translatable in Odest quando:
 - Il suo vocabolario è totalmente mappabile in quello di Odest
 - l'assiomatizzazione di Osource vale in Odest
 - Non c'è perdita di informazione
 - Non si introducono inconsistenze

Strongly-Translatable, esempio



Osource è strongly-translatable in Odest usando il mapping:
Fruit → Fruit, Apple → Pomme, Pear → Poire, Citrus → Agrume, Lemon → Citron,
Orange → Orange

Non si ha perdita di informazione. Si noti però che in Osource non vi è un concetto equivalente a Pamplemousse

Approx-Translatable

- Osource è **Approx-Translatable** in Odest quando è **Weakly-Translatable** e possono essere introdotte delle inconsistenze
- Tipicamente questo accade perché alcuni concetti che dovrebbero corrispondere l'uno con l'altro risultano affini ma non identici
- **Esempio:**
Il coriandolo, a seconda della tradizione culinaria considerata, viene visto come affine al prezzemolo (là dove se ne usano le foglie) o al pepe (là dove se ne utilizzano i semi). È la stessa pianta ma a seconda dell'ontologia di riferimento potrà avere proprietà (ontologiche) differenti



Applicazioni

- **Beni culturali** (es. per risorse documentali
<http://www.sparontologies.net/ontologies>)
- **Musei**
- **Applicazioni legali**
- **Applicazioni mediche** (es. Genoma, moltissime sono collezionate in
<https://bioportal.bioontology.org/ontologies>)
- **Applicazioni geografiche**
- **Applicazioni meteorologiche** (es.
http://cmapspublic3.ihmc.us/rid=1040074543812_1046692348_15282/SupercellThunderstorms.cmap)
- **Curricula di studi transnazionali**
- ...

OWL2 e DB: considerazioni

- I documenti OWL 2 conservano informazione ma nonostante ciò **OWL 2 non è un framework per basi di dati**
- Parte della terminologia evoca assonanze con i DB ma la **semantica sottostante è differente**:
 - un fatto non contenuto in un **DB** è considerato **falso** (**closed world assumption**) in **OWL 2** sarà **mancante** (**open world assumption**)
 - OWL **non richiede** che le uniche proprietà di un individuo siano quelle della classe a cui appartiene
 - **Classi e proprietà** possono avere **definizioni multiple**
 - Le informazioni che riguardano un individuo possono essere **distribuite in documenti diversi**

Ulteriori considerazioni

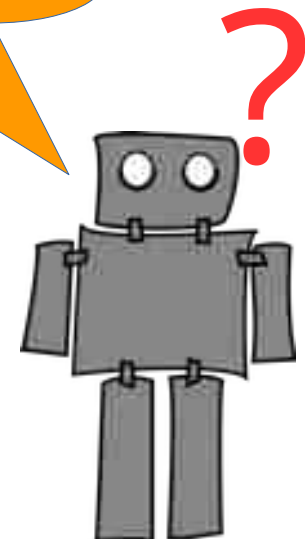
- OWL 2 non è un linguaggio di programmazione, è un **linguaggio di rappresentazione dichiarativo**
- Sulla conoscenza rappresentata in OWL 2 è possibile applicare dei programmi per effettuare del reasoning e rispondere a query
- **Is-a e Part-of non sono sufficienti per rappresentare qualsiasi tipo di conoscenza**



Rappresentare le azioni

“Le azioni non sono rappresentabili in termini di relazioni Is-a e Part-of”

L'inferenza ontologica
non basta per attraversare
la strada ...



Planning

- Molte applicazioni dei sistemi di inferenza concernono il **decidere quali azioni eseguire**, tipicamente per raggiungere un obiettivo
- **Pianificare** significa costruire una sequenza di azioni per soddisfare un certo fine
- Un **problema di pianificazione** include la specifica degli elementi di interesse del mondo, delle azioni a disposizione, degli obiettivi

Planning

- Il mondo è descritto da un insieme di variabili: tipicamente è espresso nel linguaggio **PDDL** (Planning Domain Definition Language)
- Uno stato è una **congiunzione di atomi ground**, in cui non compaiono funzioni, esempio: *At(Camion1, Bari) $\wedge$ At(Camion2, Lecce)*
- Le azioni sono descritte in maniera schematica e hanno un impatto limitato sul mondo
- In generale, in un certo stato solo un sottoinsieme delle azioni sarà applicabile e di queste solo una sarà applicata
- Se l'azione scelta viene applicata realmente subito nel mondo reale potrebbe non esserci possibilità di backtracking

Azioni – Situation Calculus

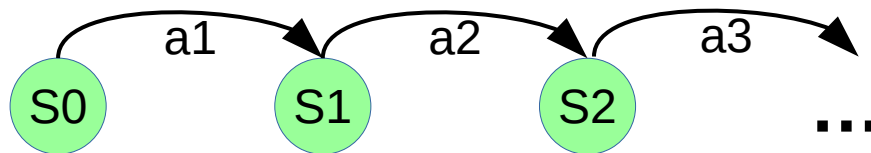
- La rappresentazione e il ragionamento su azioni si basa spesso sul **Situation Calculus**, una rappresentazione logica che identifica i concetti base di:
 - **Azione**: qualcosa che viene compiuto e influenza il mondo
 - **Situazione**: stati derivanti dall'esecuzione di qualche azione
 - **Fluente**: proprietà che può cambiare valore (fluire)
 - **Predicato atemporale** (eterno): sono funzioni o predicati il cui calcolo non è influenzato dalle azioni

- Relazione o proprietà che può cambiare valore con l'esecuzione di azioni
- Viene specificata fra i suoi parametri la **situazione**, esempi:
 - **Adjacent(R1, R2, s)**: R1 e R2 sono adiacenti nella situazione s
 - **Holds(At(R, Loc), s)**: R si trova in posizione Loc nella situazione s

- Rappresenta qualcosa che viene compiuto
- In un contesto mono-agente non occorre indicare chi sia l'attore
 - **Esempio:** Move(R, L1, L2)
rappresenta l'azione che sposta R da L1 a L2.
 - NB: In logica questo non è un predicato bensì è una funzione, restituisce un oggetto del dominio di riferimento
- **Quindi un'azione è intesa come un oggetto intangibile**, prodotto da una funzione (nell'esempio ternaria)

Legare situazioni e azioni

- Nel situation calculus il tempo non è espresso in modo esplicito ma è comunque scandito dalla sequenza degli eventi
- Si parte da una **situazione iniziale S0** e si applica una sequenza di **eventi** generando successivamente le **situazioni S1, S2**, ecc.



Legare situazioni e azioni

- **Do(Azioni, S)**: funzione che restituisce la situazione raggiunta, applicando la sequenza di azioni indicate a partire dallo stato indicato:
 - **Do([], s) = s**
 - **Do([a | rimanenza], s) =
Do(rimanenza, Risultato(a, s))**
- **NOTA**: due situazioni sono identiche esclusivamente se sono originate dallo stesso stato iniziale applicando la stessa sequenza di azioni. In altri termini una situazione è identificata dalla storia che l'ha prodotta

$$[\text{Do}(\text{Azioni1}, S1) = \text{Do}(\text{Azioni2}, S2)] \\ \Leftrightarrow [(\text{Azioni1} = \text{Azioni2}) \wedge (S1 = S2)]$$

Legare situazioni e azioni

- **Do(Azioni, S):** tramite questa funzione un agente può fare **proiezione**, cioè può ragionare sugli effetti delle azioni, in particolare può:
 - **verificare** se un corso d'azione attraversa **situazioni che godono di determinate proprietà**
 - **pianificare un corso d'azione:** quale sequenza di azioni permette di raggiungere **una situazione che gode di una specifica proprietà?**

Ragionare su azioni: esempi

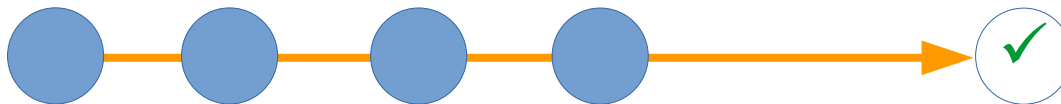
- **Proprietà che deve essere soddisfatta da tutte le situazioni attraversate:**

Un trasportatore deve portare merci in diverse località. A ogni consegna può scegliere fra raggiungere la località successiva oppure fare rifornimento. Non deve mai rimanere a secco



- **Proprietà finale (goal):**

Un ciclista deve comporre le diverse parti di una bicicletta per costruirla



Rappresentare le azioni

- Dobbiamo descrivere in logica anche le azioni
- Azione descritta da **ASSIOMI DI APPLICABILITÀ** e **ASSIOMI DI EFFETTO**:

Assioma di Applicabilità:

$\forall \text{params}, s \quad \text{Applicable}(\text{Action}(\text{params}), s) \Leftrightarrow \text{Precond}(\text{params}, s)$

definisce che un'azione può (fisicamente) essere applicata in una situazione se e solo se valgono determinate precondizioni

- Applicable è un nuovo **predicato** che lega un'azione a una situazione
- Params è un **insieme di oggetti**
- Action(params) indica l'**applicazione dell'azione agli oggetti**
- Precond è una **formula** che rappresenta le precondizioni dell'azione

Esempio

- **ASSIOMA DI APPLICABILITÀ**

$\forall \text{params}, s \quad \text{Applicable}(\text{Action}(\text{params}), s) \Leftrightarrow \text{Precond}(\text{params}, s)$

- **Esempio:** $\text{Applicable}(\text{go}(X,Y),S) \Leftrightarrow \text{At}(X, S) \wedge \text{Adjacent}(X, Y)$

- $\text{go}(X, Y)$: azione, andare dalla posizione X alla posizione Y
- Params è l'insieme costituito dalle variabili X e Y
- $\text{At}(\text{Agente}, X, S) \wedge \text{Adjacent}(X, Y)$ è la precondizione. $\text{Go}(X, Y)$ può essere eseguita a patto che l'agente sia in X (fluente) e che X sia adiacente a Y (predicato atemporale)

Rappresentare le azioni

- **ASSIOMI DI EFFETTO:**

$\forall \text{params}, s \text{ Applicable}(\text{Action}(\text{params}), s) \Rightarrow$
 $\text{Effects}(\text{params},$

$\text{Result}(\text{Action}(\text{params}), s))$

- specifica gli effetti di un'azione sulla situazione in cui è eseguita
- Effects è una **formula** vera nello stato risultante dall'esecuzione dell'azione
- Result è una **funzione** che denota lo stato in cui si va eseguendo l'azione in s

Rappresentare le azioni

- **ASSIOMI DI EFFETTO:**

$\forall \text{params}, s \text{ Applicable}(\text{Action}(\text{params}), s) \Rightarrow$
 $\text{Effects}(\text{params},$

$\text{Result}(\text{Action}(\text{params}), s))$

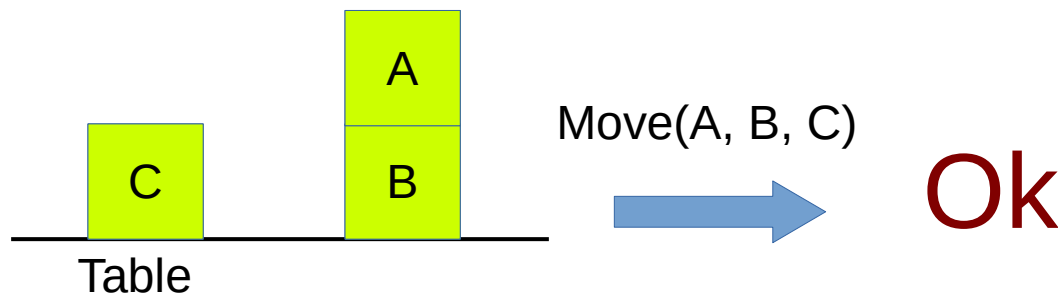
- specifica gli effetti di un'azione sulla situazione in cui è eseguita
- **Esempio:** $\text{Applicable}(\text{go}(X, Y), S) \Rightarrow \text{At}(Y, \text{Result}(\text{go}(X, Y), S))$
- L'effetto dell'azione applicabile $\text{go}(X, Y)$ è che nella situazione risultante dall'esecuzione di tale azione in s (identificata da $\text{Result}(\text{go}(X, Y), S)$) l'agente (sottinteso nell'esempio in quanto unico) si trova alla posizione Y

Esempio: mondo dei blocchi

- **Move(X, Y, Z):** sposta X da Y a Z
- **Assioma di applicabilità:**
 - $\forall x,y,z,s \text{ Applicable}(\text{Move}(x,y,z), s) \Leftrightarrow \text{Clear}(x, s) \wedge \text{Clear}(z, s) \wedge \text{On}(x,y,s) \wedge x \neq z \wedge y \neq z \wedge x \neq \text{Table}$

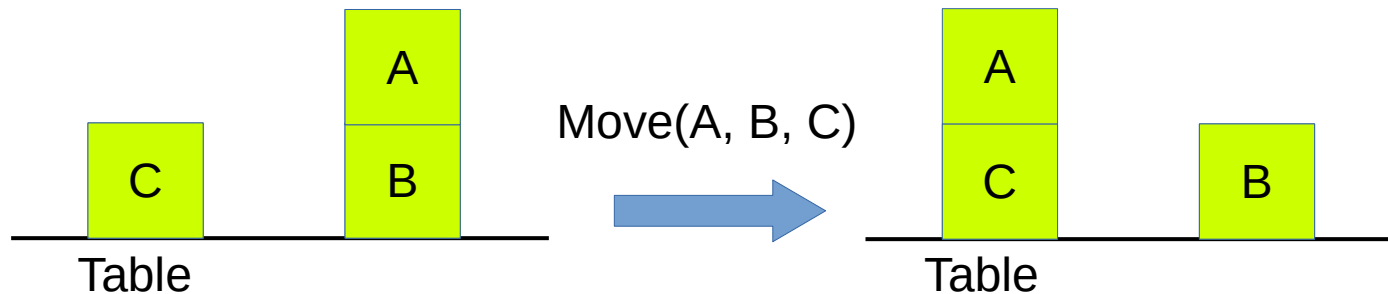
quindi

- $\text{Move}(A,B,A)$: no, non posso spostare un blocco sopra a se stesso
- $\text{Move}(A,B,B)$: no, non denota uno spostamento



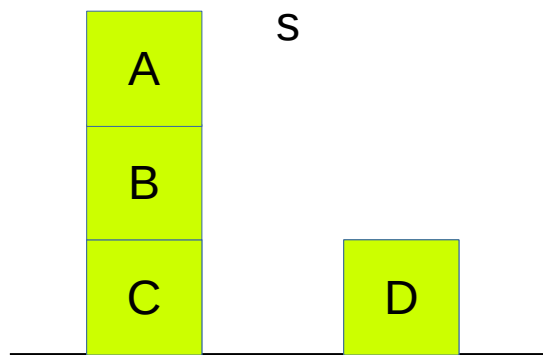
Esempio: mondo dei blocchi

- **Move(X, Y, Z):** sposta X da Y a Z
- **Assioma di effetto:**
 - $\forall x,y,z,s \text{ Applicable}(\text{Move}(x,y,z), s) \Rightarrow \text{On}(x,z, \text{Result}(\text{Move}(x,y,z), s)) \wedge \text{clear}(y, \text{Result}(\text{Move}(x,y,z), s))$

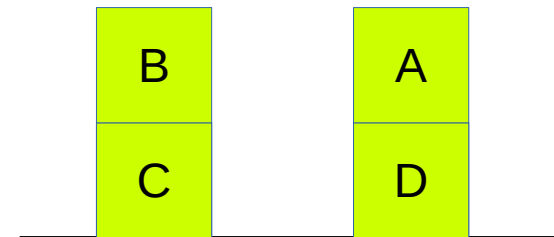


Inferenza

- Da (1) conoscenza di stato iniziale, (2) assiomi di applicabilità e (3) assiomi di effetto (KB nel seguito) è possibile derivare fatti



Result(Move(A, B, D), s)

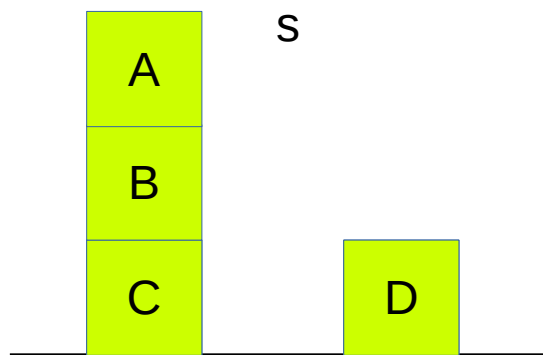


È possibile derivare che:
 $\text{On}(A, D, \text{Result}(\text{Move}(A, B, D), s))$
 $\text{Clear}(B, \text{Result}(\text{Move}(A, B, D), s))$

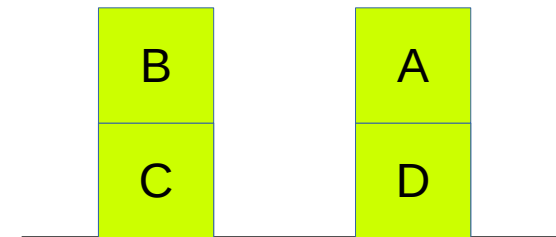
Per definizione di Move

Frame problem

- Dalla conoscenza di stato iniziale, assiomi di applicabilità e assiomi di effetto (KB nel seguito) **NON** è possibile derivare tutti i fatti che ci aspettiamo



$\text{Result}(\text{Move}(A, B, D), s)$

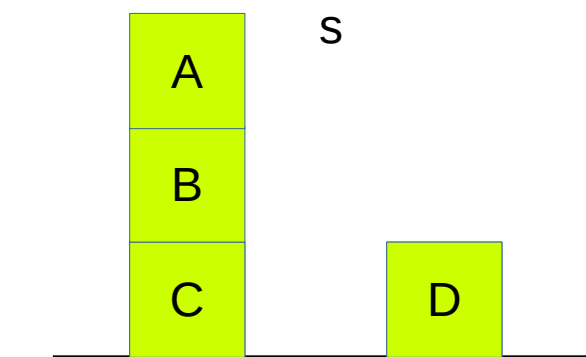


NON è possibile derivare che:
 $\text{On}(B, C, \text{Result}(\text{Move}(A, B, D), s))$

Posso ragionare su cosa è cambiato ma non su cosa non è cambiato

Frame problem

- Dalla conoscenza di stato iniziale, assiomi di applicabilità e assiomi di effetto (KB nel seguito) **NON** è possibile derivare tutti i fatti che ci aspettiamo

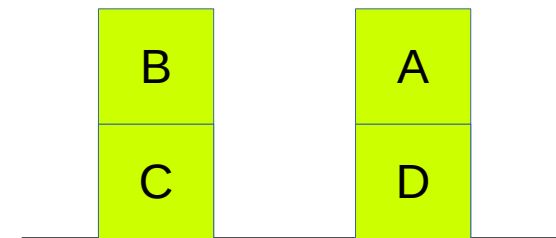


So che vale: $\text{On}(B, C, s)$

$\text{Move}(A, B, D)$



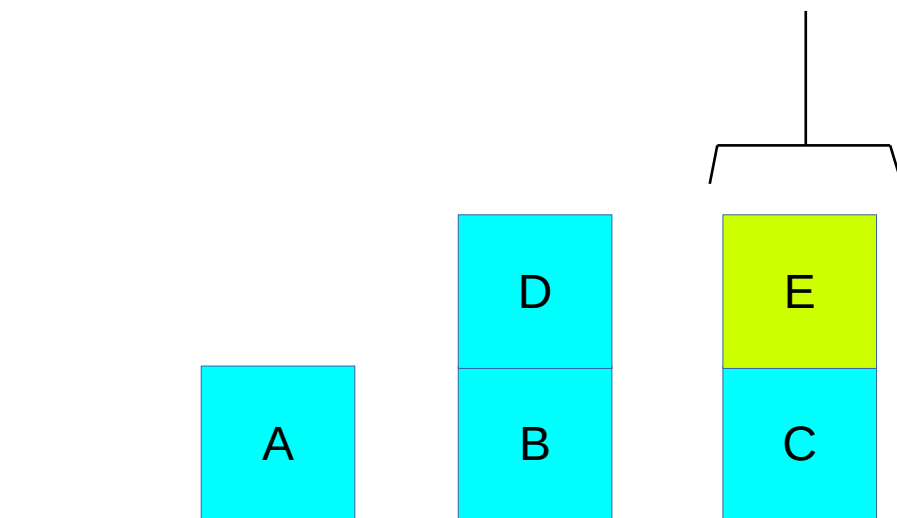
$s' = \text{Result}(\text{Move}(A, B, D), s)$



Abbiamo conoscenza su s ma senza assiomi che permettano di fare inferenza il sistema non può sapere se ciò che non è stato modificato da $\text{Move}(\dots)$ vale anche in s' . Bisogna “dirglielo” in qualche modo.

Frame problem

- **Frame problem (problema della cornice):**
normalmente le azioni hanno un impatto limitato: come rappresentare ciò che non viene modificato da un'azione?



Quando il braccio afferra E
l'unica cosa che cambia è che il
braccio non è più vuoto. Per il resto
la situazione rimane immutata



Il sistema automatico deve poter inferire
cosa S1 eredita "as-is" da S

Frame problem 1: enumerare ciò che non cambia

- **Frame problem (problema della cornice):**
normalmente le azioni hanno un impatto limitato: come rappresentare ciò che non viene modificato da un'azione?

1) Rappresentazione esplicita tramite **assiomi di frame:**

$$\forall \text{params, vars, s } \text{fluent}(\text{vars}, s) \wedge \text{params} \neq \text{vars} \Rightarrow \\ \text{fluent}(\text{vars}, \text{Result}(\text{Action}(\text{params}), s))$$

Per ogni azione viene definito un assioma di questo tipo per ogni fluente

Frame problem 1: enumerare ciò che non cambia

- Rappresentazione esplicita tramite **assiomi di frame**:
 $\forall \text{params, vars, s } \text{fluent}(\text{vars}, s) \wedge \text{params} \neq \text{vars} \Rightarrow$
 $\text{fluent}(\text{vars}, \text{Result}(\text{Action}(\text{params}), s))$

Esempio: azione: Move, vars = {x, y}, params = {z, w}

$\forall \text{params, vars, s } \text{on}(\mathbf{x}, \mathbf{y}, \mathbf{s}) \wedge x \neq z \Rightarrow$
 $\text{on}(x, y, \text{Result}(\mathbf{Move}(\mathbf{z}, \mathbf{w}), s))$

$\forall \text{params, vars, s } \text{clear}(\mathbf{x}, \mathbf{s}) \wedge x \neq w \Rightarrow$
 $\text{clear}(x, \text{Result}(\mathbf{Move}(\mathbf{z}, \mathbf{w}), s))$

Frame problem

- **Problema:**

occorre introdurre frame axioms per **tutti** i fluenti che non sono modificati da ciascuna azione!

- **Esempio**

Se i blocchi ora possono essere dipinti di un colore, rivestiti di un materiale, ecc. occorrerà specificare frame axiom per ciascuna di queste proprietà:

$\forall \text{params, vars, s colore}(x, y, s) \wedge x \neq z \Rightarrow$
 $\text{colore}(x, y, \text{Result}(\text{Move}(z, w), s))$

$\forall \text{params, vars, s materiale}(x, y, s) \wedge x \neq z \Rightarrow$
 $\text{materiale}(x, y, \text{Result}(\text{Move}(z, w), s))$

...

Frame problem 2: evitare l'enumerazione

- introduciamo un modo per dire al sistema inferenziale che ciò che non è specificamente espresso come effetto è inteso rimanere immutato

Frame problem

- **ASSIOMA DI STATO SUCCESSORE:**

**Azione applicabile $\Rightarrow$ (fluente vero nella situazione risultante
 $\Leftrightarrow$ (l'azione lo rende vero $\vee$
era vero e l'azione non l'ha reso falso))**

- Questo assioma esprime il modo in cui le **situazioni si evolvono** l'una dall'altra a seguito dell'esecuzione delle azioni. In particolare dice quali parti di una situazione sono ereditati dalla precedente

Unique action names

- Il situation calculus è completato da assiomi che catturano:
 - che azioni con nomi diversi sono diverse:
 $A_i(x, \dots) \neq A_j(y, \dots)$
 - E per ogni nome di azione, che due usi di quel nome sono identici se e solo se gli argomenti sono identici:
 $A(x_1, \dots, x_n) = A(y_1, \dots, y_n) \Rightarrow x_1 = y_1 \wedge \dots \wedge x_n = y_n$